Web Application QA & Debug Report

Author: Kumar Kaushik | Target: Conduit - RealWorld Angular live demo (https://demo.realworld.show) | Status: FINAL - findings approved 10 November 2026

Target Application & Scope: Live angular frontend at https://demo.realworld.show with session-isolated backend. Evaluated core user flows: registration, authentication, article authoring, editing, deletion, tag filtering, commenting, keyboard accessibility (WCAG 2.2), and API edge cases. All 5 issues reproduced under controlled test conditions.

5. Bug Report Table (Assessment Template Compliance)

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1 (QA-01)	Rapid repeated favorite clicks leave the visible count and state incorrect until reload	1. Sign in with a controlled account and create a disposable article. 2. On the Global Feed, confirm its favorite count is 0 and the button is visually unfavorited. 3. Double-click its favorite button. 4. Observe the immediate UI. 5. Reload the feed. 6. Reset with one click and repeat the double-click sequence.	**Exp:** The interface should either ignore additional clicks while the request is in flight or accurately show the persisted favorite state and count without requiring a reload. **Act:** After the double-click, the feed still showed 0 and the outline (unfavorited) style. Reloading showed 1 and the filled (favorited) style. The same immediate-versus-reloaded mismatch was reproduced after resetting and repeating the sequence.	**Medium**	Rapid requests are likely accepted while the UI is pending. The inspected public component uses a CSS disabled class during submission rather than a native disabled attribute, but the exact deployed source revision is not established.
2 (QA-02)	Deleting an already-deleted article leaves a stale tab stuck without feedback	1. Open the same controlled disposable article in two tabs. 2. Delete it in tab B and confirm that tab B returns to the feed. 3. Without refreshing tab A, click Delete Article. 4. Repeat with a fresh controlled article.	**Exp:** The stale tab should show an understandable not-found/deletion error and recover to the feed, or refresh its state. **Act:** The stale tab remains on the already-deleted article. Its delete controls gain a disabled CSS class, no feedback or recovery path is shown, and the browser captures an error object. The native disabled property remains false. The behavior was reproduced with a second disposable article.	**Medium**	The current public article component sets its deleting state and handles only the success callback, which supports a missing-error-recovery hypothesis. The deployed build revision is not confirmed, so this is not asserted as a proven source-line cause.
3 (QA-03)	Missing article leaves an unexplained blank content area	1. Navigate directly to /article/oytra-qa-nonexistent-20261110. 2. Wait for the request to settle. 3. Observe the main content area. 4. Repeat while logged in and logged out.	**Exp:** Show an article-not-found message and a recovery action such as Return to feed. **Act:** Only the site navigation and footer remain. No not-found or retry message appears. A separate unauthenticated request to the matching API endpoint returned HTTP 404 and errors.article = [not found].	**Medium**	A frontend error handling or rendering problem is suspected. The API itself correctly reports 404. Current public source includes error handling, so its exact relationship to the deployed build is unconfirmed; do not claim a specific source line is proven responsible.
4 (QA-04)	Comment delete action cannot be reached by keyboard	1. Open your own article containing your own comment. 2. Focus the comment-author link. 3. Press Tab. 4. Inspect the delete icon's element and accessibility role. 5. Compare with a mouse click on that icon.	**Exp:** Tab reaches a named Delete comment button; Enter or Space activates it. **Act:** Tab moves from the comment-author link directly to the footer Conduit link, skipping the trash icon. The control is an i element inside a span, has tabIndex -1 and no role or accessible label. A mouse click deleted the disposable comment successfully.	**Medium**	The deployed DOM uses a non-focusable decorative element as the interactive control. This immediate markup cause was inspected directly; it does not require a backend hypothesis.
5 (QA-05)	Whitespace-only comments are accepted and persisted	1. Sign in with a controlled account and open its article. 2. Enter three spaces in Write a comment. 3. Click Post Comment. 4. Reload the article.	**Exp:** Reject a comment with no non-whitespace content and display a clear validation message. **Act:** A visually empty comment card is created with author/date/delete metadata. It remains after refresh. The saved rendered paragraph contains only spaces.	**Low**	Missing whitespace normalization/non-empty validation is suspected. Persistence is verified through a browser refresh; backend implementation was not inspected.

Verified Passing Baseline Checks: Verified: home/feed/tags, required authentication fields, malformed email message, valid account creation/login, invalid password rejection, logout, authenticated refresh, logged-out editor/settings redirects, article creation/editing/persistence, Unicode and Markdown rendering, required article-field errors, and mouse deletion of the controlled comment.

QA-01 — Rapid repeated favorite clicks leave the visible count and state incorrect until reload

Severity: Medium | **Category:** Functional / Accessibility | **Target:** Conduit Web App

Steps to Reproduce: 1. Sign in with a controlled account and create a disposable article. 2. On the Global Feed, confirm its favorite count is 0 and the button is visually unfavorited. 3. Double-click its favorite button. 4. Observe the immediate UI. 5. Reload the feed. 6. Reset with one click and repeat the double-click sequence.

Expected Behaviour: The interface should either ignore additional clicks while the request is in flight or accurately show the persisted favorite state and count without requiring a reload.

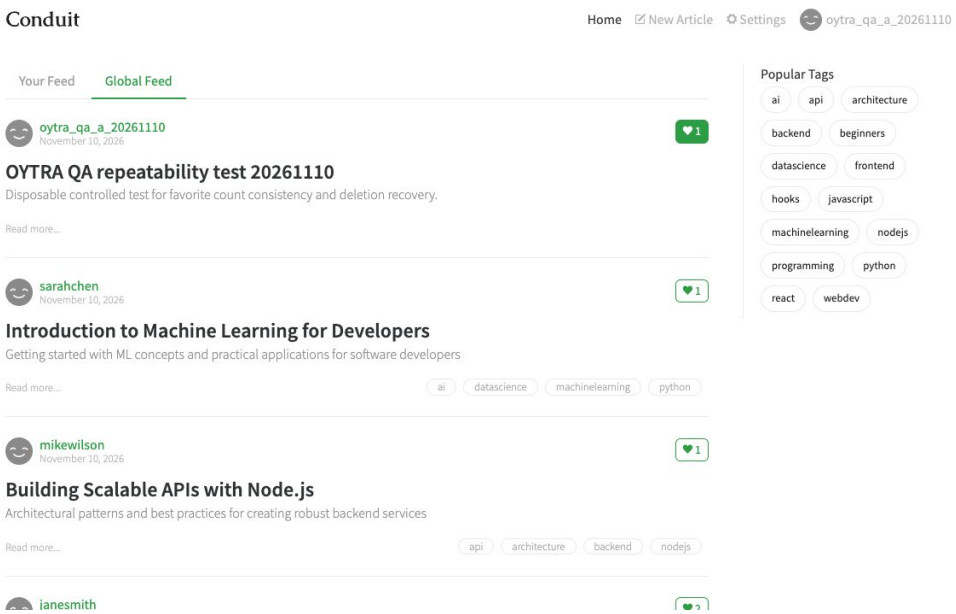
Actual Behaviour: After the double-click, the feed still showed 0 and the outline (unfavorited) style. Reloading showed 1 and the filled (favorited) style. The same immediate-versus-reloaded mismatch was reproduced after resetting and repeating the sequence.

User Impact: A user cannot tell whether their favorite action succeeded, and the count/state displayed during the session is incorrect.

Suspected / Confirmed Cause: Rapid requests are likely accepted while the UI is pending. The inspected public component uses a CSS disabled class during submission rather than a native disabled attribute, but the exact deployed source revision is not established.

Recommended Remediation: Prevent repeat activation while a favorite request is pending, serialize or cancel duplicate requests, and reconcile the displayed count/state with the confirmed server result.

Attached Evidence Artifacts: favorite-doubleclick-before-refresh.jpg, favorite-doubleclick-after-refresh.jpg, favorite-repeat.json



QA-02 — Deleting an already-deleted article leaves a stale tab stuck without feedback

Severity: Medium | **Category:** Functional / Accessibility | **Target:** Conduit Web App

Steps to Reproduce: 1. Open the same controlled disposable article in two tabs. 2. Delete it in tab B and confirm that tab B returns to the feed. 3. Without refreshing tab A, click Delete Article. 4. Repeat with a fresh controlled article.

Expected Behaviour: The stale tab should show an understandable not-found/deletion error and recover to the feed, or refresh its state.

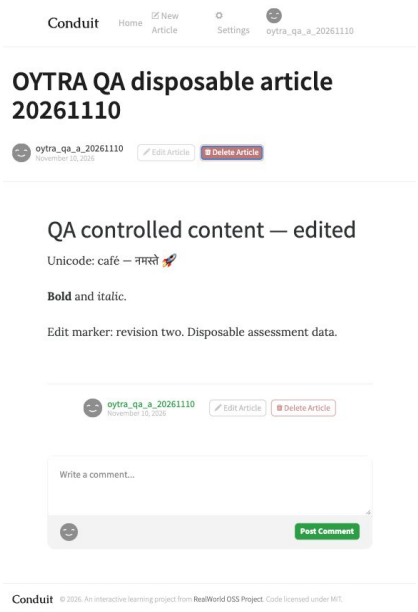
Actual Behaviour: The stale tab remains on the already-deleted article. Its delete controls gain a disabled CSS class, no feedback or recovery path is shown, and the browser captures an error object. The native disabled property remains false. The behavior was reproduced with a second disposable article.

User Impact: A user working from a stale tab is stranded on content that no longer exists and receives no indication of what happened or how to recover.

Suspected / Confirmed Cause: The current public article component sets its deleting state and handles only the success callback, which supports a missing-error-recovery hypothesis. The deployed build revision is not confirmed, so this is not asserted as a proven source-line cause.

Recommended Remediation: Handle delete failures by resetting pending state, showing a clear deleted/not-found message, and navigating or offering a route back to the feed.

Attached Evidence Artifacts: stale-delete-dom.txt, stale-delete.jpg



QA-03 — Missing article leaves an unexplained blank content area

Severity: Medium | **Category:** Functional / Accessibility | **Target:** Conduit Web App

Steps to Reproduce: 1. Navigate directly to /article/oytra-qa-nonexistent-20261110. 2. Wait for the request to settle. 3. Observe the main content area. 4. Repeat while logged in and logged out.

Expected Behaviour: Show an article-not-found message and a recovery action such as Return to feed.

Actual Behaviour: Only the site navigation and footer remain. No not-found or retry message appears. A separate unauthenticated request to the matching API endpoint returned HTTP 404 and errors.article = [not found].

User Impact: Users following a stale or mistyped article link cannot tell whether the article is missing or the application is still loading.

Suspected / Confirmed Cause: A frontend error handling or rendering problem is suspected. The API itself correctly reports 404. Current public source includes error handling, so its exact relationship to the deployed build is unconfirmed; do not claim a specific source line is proven responsible.

Recommended Remediation: Ensure the deployed article view maps 404 responses to a visible error state and provides a route back to the feed.

Attached Evidence Artifacts: missing-article-dom.txt, missing-article.jpg, missing-article-api.json



QA-04 — Comment delete action cannot be reached by keyboard

Severity: Medium | **Category:** Functional / Accessibility | **Target:** Conduit Web App

Steps to Reproduce: 1. Open your own article containing your own comment. 2. Focus the comment-author link. 3. Press Tab. 4. Inspect the delete icon's element and accessibility role. 5. Compare with a mouse click on that icon.

Expected Behaviour: Tab reaches a named Delete comment button; Enter or Space activates it.

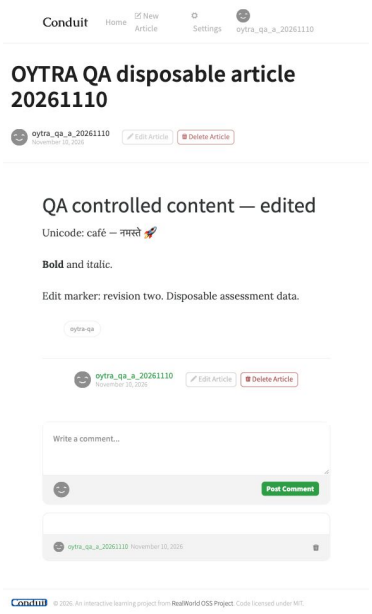
Actual Behaviour: Tab moves from the comment-author link directly to the footer Conduit link, skipping the trash icon. The control is an i element inside a span, has tabIndex -1 and no role or accessible label. A mouse click deleted the disposable comment successfully.

User Impact: Keyboard-only users cannot invoke the visible comment-delete action.

Suspected / Confirmed Cause: The deployed DOM uses a non-focusable decorative element as the interactive control. This immediate markup cause was inspected directly; it does not require a backend hypothesis.

Recommended Remediation: Replace it with a native button with an accessible Delete comment label; keep the icon decorative and preserve the existing deletion handler.

Attached Evidence Artifacts: comment-keyboard-skip.json, comment-keyboard-skip.jpg, blank-comment-markup.html



QA-05 — Whitespace-only comments are accepted and persisted

Severity: Low | **Category:** Functional / Accessibility | **Target:** Conduit Web App

Steps to Reproduce: 1. Sign in with a controlled account and open its article. 2. Enter three spaces in Write a comment. 3. Click Post Comment. 4. Reload the article.

Expected Behaviour: Reject a comment with no non-whitespace content and display a clear validation message.

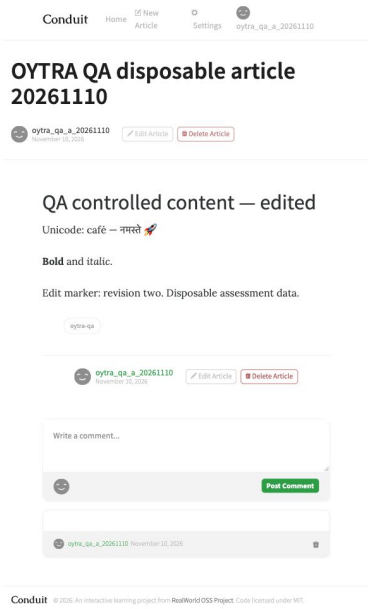
Actual Behaviour: A visually empty comment card is created with author/date/delete metadata. It remains after refresh. The saved rendered paragraph contains only spaces.

User Impact: Users can create empty discussion entries, cluttering the thread without meaningful content.

Suspected / Confirmed Cause: Missing whitespace normalization/non-empty validation is suspected. Persistence is verified through a browser refresh; backend implementation was not inspected.

Recommended Remediation: Trim or validate non-whitespace content on the server and mirror that validation in the comment form.

Attached Evidence Artifacts: blank-comment-markup.html, blank-comment-before-refresh.jpg, blank-comment-persisted.jpg



Root-Cause Analysis — In-Depth Investigation (QA-05)

QA-04 prevents keyboard users from deleting their own comments. The problem occurs when a comment owned by the signed-in user displays its trash icon. In the live DOM, that control is an i element nested in a span, with tabIndex -1 and no button role or accessible label. Pressing Tab from the comment-author link moved focus directly to the footer link, skipping deletion. Clicking the same trash icon with the mouse successfully deleted the disposable comment, showing that the deletion path itself worked in this test. The immediate cause is therefore confirmed at the rendered-markup level: the interactive action lacks native keyboard semantics and focusability. Replace the control with a type=button element carrying an accessible Delete comment label and retain the existing handler, while hiding the decorative icon from assistive technology. Verify the fix by reaching the button with Tab, activating it separately with Enter and Space, checking a descriptive accessible name, and confirming that only the controlled user's intended comment disappears.

Investigation Scope, Limitations & Excluded Observations

This is a time-boxed live-demo review, not a comprehensive security audit. The backend explicitly isolates accounts/sessions; re-registration created a different context, while correct login restored the original article. This behavior is excluded from defects. A controlled two-account authorization investigation produced inconsistent ownership context after the demo session changed, so it is deliberately excluded rather than overstated as a security finding. A direct account-A ownership check correctly created and deleted an article as account A. Long-text limits, session expiration and a reliable mobile screenshot remain incomplete. No full screen-reader session was performed. The old demo.realworld.io HTTP 404 is an outdated reference, not an application defect. Current public source was inspected only selectively; the deployed commit is unknown. No application fixes were required or made.

Technical References & Standards Grounding

• https://github.com/realworld-apps/angular-realworld-example-app
• https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html
• https://www.w3.org/WAI/WCAG22/Understanding/keyboard.html